1	2	3	$\sum$

Algorithmen, Datenstrukturen und Datenabstraktion (WiSe 18/19)

Docent: Prof. Dr. Wolfgang Mulzer

Tutorin: Valeria Zahoransky; Tutorial: 07

Exercise 06

Release: 19.11.2018; Due: 30.11.2018

Viktoriya Kraleva, Samuel Enderwitz

---

1 Further Heap-Operations (__/10)

(a) *Write code for a binary heap with n elements, stored as an array. Also, implement the following two operations, with worst-case running time $O(\log n)$.*

- ***decreaseKey***(i, k)*: Set the key at position i to k and restore the heap property (Precondition: k is less than or equal to the old key at i).*
- ***delete***(i)*: Remove the element at position i and restore the heap property.*

*Both **decreaseKey** and **delete** may use **bubble-up** and **bubble-down**. Briefly justify correctness and running time of your implementation.*

```java
package de.smuu.add.e03;

public interface BinaryHeapInterface<E extends Comparable<E>> {

    void insert(E element);
    E get(int i) throws IndexOutOfBoundsException;
    E delete(int i) throws IndexOutOfBoundsException;
    E getRoot() throws HeapEmptyException;
    boolean isEmpty();
    int getSize();
    void bubble_up(int i);
    void bubble_down(int i);
    void decreaseKey(int i, E element) throws IndexOutOfBoundsException;
    int getParent(int i) throws IndexOutOfBoundsException;
    int getLeftChild(int i) throws IndexOutOfBoundsException;
    int getRightChild(int i) throws IndexOutOfBoundsException;
}
```

Listing 1: BinaryHeapInterface

```java
package de.smuu.add.e03;

public class BinaryHeap<E extends Comparable<E>> implements
    BinaryHeapInterface<E> {

    private DynamicArray<E> dA;

```

```java
    public BinaryHeap() {
        this(20);
    }

    public BinaryHeap(E[] array) {
        dA = new DynamicArray<>(array);
        buildHeap();
    }

    public BinaryHeap(int size) {
        dA = new DynamicArray<>(size);
    }

    private void buildHeap() {
        buildHeap(0);
    }

    private void buildHeap(int start) {
        for (int i = getSize()-1; i > start; i--) {
            bubble_down(i);
        }
    }

    @Override
    public void insert(E element) {
        dA.insert(element);
        if (getSize() != 1)
            bubble_up(getSize() - 1);
    }

    @Override
    public E get(int i) throws IndexOutOfBoundsException {
        if (i >= dA.getSize()) {
            throw new IndexOutOfBoundsException(i + ">=" + dA.getSize())
;
        }
        return dA.get(i);
    }

    @Override
    public E delete(int i) throws IndexOutOfBoundsException {
        E element = get(i);
        dA.delete(i);
        buildHeap(i);
        return element;
    }

    @Override
    public E getRoot() throws HeapEmptyException {
        return get(0);
    }

    @Override
    public boolean isEmpty() {
        return getSize() == 0;
    }

```

```java
    @Override
    public int getSize() {
        return dA.getSize();
    }

    @Override
    public void bubble_up(int i) {
        if (i == 0)
            return;
        if (dA.get(i).compareTo(dA.get(getParent(i))) < 0) {
            dA.switchPos(i, getParent(i));
            bubble_up(getParent(i));
        }
    }

    @Override
    public void bubble_down(int i) {
        E thisEl = dA.get(i);
        E left = dA.get(getLeftChild(i));
        E right = dA.get(getRightChild(i));
        int min = 0;
        if (getLeftChild(i) <= getSize() && left.compareTo(thisEl) < 0) {
            min = getLeftChild(i);
        }
        if (getRightChild(i) <= getSize() && right.compareTo(thisEl) < 0 && right.compareTo(left) < 0) {
            min = getRightChild(i);
        }
        if (min == 0) {
            return;
        }
        dA.switchPos(i, min);
        bubble_down(min);
    }

    @Override
    public void decreaseKey(int i, E element) throws IndexOutOfBoundsException {
        if (element.compareTo(get(i)) < 0) {
            dA.insert(i, element);
            bubble_up(i);
        }
    }

    @Override
    public int getParent(int i) throws IndexOutOfBoundsException {
        if (i == 0) {
            throw new IndexOutOfBoundsException("you already at root");
        }
        return (int) Math.floor(((double) (i - 1)) / 2);
    }

    @Override
    public int getLeftChild(int i) throws IndexOutOfBoundsException {
        if (2 * i + 1 >= getSize()) {
            throw new IndexOutOfBoundsException(i + "has no right child"
```

```
    );
        }
        return 2 * i + 1;
    }

    @Override
    public int getRightChild(int i) throws IndexOutOfBoundsException {
        if (2 * i + 2 >= getSize()) {
            throw new IndexOutOfBoundsException(i + "has no right child"
    );
        }
        return 2 * i + 2;
    }

    @Override
    public String toString() {
        //TODO: where is the end?
        return dA.getSize() + dA.toString();
    }
}
```

Listing 2: BinaryHeap

```
package de.smuu.add.e03;

public class HeapEmptyException extends RuntimeException {

    public HeapEmptyException(String string) {
        super(string);
    }
}
```

Listing 3: HeapEmptyException

```
package de.smuu.add.e03;

import java.util.Arrays;

public class DynamicArray<E> {

    private E[] array;
    private int pointer;

    public DynamicArray() {
        this(20);
    }

    public DynamicArray(E[] array) {
        this.array = array;
        this.pointer = array.length;
    }

    public DynamicArray(int size) {
        this.array = (E[]) new Object[size];
        this.pointer = 0;
    }

```

```java
    public void insert(E element) {
        if (pointer == array.length) {
            resize();
        }
        insert(pointer, element);
        this.pointer++;
    }

    public void insert(int pos, E element) throws IndexOutOfBoundsException {
        if (pos > pointer) {
            throw new IndexOutOfBoundsException();
        }
        this.array[pos] = element;
    }

    public E get(int i) {
        return this.array[i];
    }

    public E delete(int i) {
        E element = get(i);
        for (int j = i+1; j < pointer; j++) {
            this.array[j-1] = this.array[j];
        }
        this.pointer--;
        return element;
    }

    public int getSize() {
        return pointer;
    }

    private void resize() {
        int newSize = getSize()*2;
        E[] newArray = (E[]) new Object[newSize];
        for (int i = 0; i < array.length; i++) {
            newArray[i] = array[i];
        }
        array = newArray;
    }

    public void switchPos(int i, int j) {
        E tmp = array[j];
        array[j] = array[i];
        array[i] = tmp;
    }

    @Override
    public String toString() {
        return Arrays.toString(array);
    }
}
```

Listing 4: DynamicArray

(b) *We would like to extend the interface PriorityQueue with the additional operations*

from (a). Explain briefly why an efficient implementation needs a "pointer" into the data structure (i.e., why is it not enough to pass only the old and the new key to decreaseKey, even if the old key appears only once?).
How would you implement this without violating the information hiding principle? Define a new interface `Position` *that represents a pointer into the priority queue. Explain briefly how to modify* `PriorityQueue` *in order to use* `Position`*. What problems does this create? Sketch a possible solution. (You do not need to write any code.)*

Anmerkung: Wir haben diese Aufgabe mit vielen anderen aus anderen Gruppen besprochen. Dadurch kann es ggf. dazu kommen, dass andere Lösungen dieser ähneln könnte.

Eine Skizze zu unserer Überlegung liegt im Anhang bei.

Wir haben eine beliebig große Schlüsselmenge. Diese Schlüsselmenge wird mit einer gut gewählten zufälligen Hashfunktion auf einen kleinen Array abgebildet. Wenn die Zielmenge groß genug ist, kommt es selten bis nie zu Kollisionen.

In diesem Array werden die Pointer abgelegt, die auf die Elemente in unserer Struktur zeigen.

insert: Dabei wird das Element in der Struktur gespeichert und der Speicherort wir mithilfe eines Pointers in den Array abgespeichert.

getMin: Nach dem herausnehmen des kleinsten Elements aus der Struktur wird der Eintrag in der Tabelle gelöscht.

delete: Über die Hash-Funktion und der Tabelle erhält man die Position des Elements. Das Element wird dann aus der Struktur entfert und die Eigenschaften der Struktur werden wieder hergestellt. Der Pointer wird aus der Tabelle gelöscht.

decreaseKey: Das Element wird wieder über die Hash-Funktion und die Tabelle gefunden. Dann wird der Wert verändert und die Eigenschaft der Struktur wiederhergestellt. In der Tabelle wird der Eintrag gelöscht und ein neuer wird angelegt.

Das Geheimnisprinzip wird erhalten, da die Implementierung der Tabelle unabhängig der genutzen Struktur ist. Der Anwender kann Elemente schnell in O(n) Zeit finden, löschen und verändern.

2 Elementary Probability Theory (__/10)

We have N bins, and we throw n balls into them such that each throw is independent of the others and such that each bin is hit with the same probability $1/N$.

(a) *Compute the probability that bin i remains empty, for $i = 1, \ldots, N$. Furthermore, let Y_i be the indicator random variable that is 1, if bin i remains empty, and 0, otherwise. Compute the expected value $E[Y_i]$ of Y_i.*

$$
\begin{aligned}
P(\text{Ein Korb wird getroffen}) &= \frac{1}{N} \\
P(\text{Ein Korb wird n mal getroffen}) &= (\frac{1}{N})^n \\
P(\text{Ein Korb wird nicht getroffen}) &= 1 - \frac{1}{N} = \frac{N-1}{N} \\
P(\text{Ein Korb wird n mal nicht getroffen}) &= 1 + (\frac{1}{N})^n = (\frac{N-1}{N})^n \\
E[Y_i] &= 1 \cdot P(\text{nicht treff}) + 0 \cdot P(\text{treff}) \\
&= 1 \cdot (\frac{N-1}{N})^n + 0 \cdot (\frac{1}{N})^n \\
&= (\frac{N-1}{N})^n
\end{aligned}
$$

(b) *Let X be the random variable that indicates the number of bins that remain empty. Compute the expected value of X using the expected values $E[Y_i]$, for $i = 1, \ldots, N$.*

$$
\begin{aligned}
E[X] &= E[\sum_{i=0}^{N} Y_i] \quad |(1) \\
&= \sum_{i=0}^{N} E[Y_i] \\
&= \sum_{i=1}^{N} (\frac{N-1}{N})^n \\
&= N((\frac{N-1}{N})^n)
\end{aligned}
$$

(1): Linearität des Erwartungswertes

(c) *Determine a good bound $f(N)$ so that the following holds: suppose we throw n balls into N bins, independently and uniformly at random. If $n \geq f(N)$, then with probability at least $1/2$ there is at least one bin with at least two balls.*
Hint*: Imagine that the n balls are thrown successively. What is the probability that the next ball ends up in an empty bin, provided that all the previous balls have ended up in an empty bin? You might find the bound $1 + x \leq e^x$ useful. It holds for all $x \in \mathbb{R}$.*

Nach der Laplace-Formel bekommt man die Wahrscheinlichkeit, wenn man die Anzahl der Ergenisse, bei denen ein Fall eintrifft durch die gesamte Anzahl der möglichen Ergebnissen teilt.

Die Wahrscheinlichkeit, dass mindestens eine Kiste von mindestens zwei Bällen getroffen wird, ist die Gegenwahrscheinlichkeit davon, dass alle Kisten einmal oder keinmal getroffen wurde.

Wahrscheinlichkeit, dass alle Kisten nicht oder einmal getroffen wurde: $\frac{N!}{(N-n)! \cdot N^n}$.

Somit ist die Wahrscheinlichkeit, dass mindestens eine Kiste von mindestens zwei Bällen getroffen wurde: $1 - \frac{N!}{(N-n)! \cdot N^n}$.

Somit ergibt sich folgende Formel:

$$\frac{1}{2} \leq 1 - \frac{N!}{(N-n)! \cdot N^n}$$

Diese Formel muss man nun auf n umstellen und erhält somit eine untere Schranke für die Anzahl der Bälle, die man werfen muss, dass die Wahrscheinlichkeit, dass mindestens eine Kiste mindestens zwei mal getroffen wurde bei $\frac{1}{2}$ steht.

3 Hashing (__/10)

(a) *Successively insert the keys 5, 28, 19, 15, 20, 33, 12, 17, 10 into a hash table with 9 positions. Use the hash function* $h(k) = k \bmod 9$*, and resolve conflicts with chaining.*

(b) *Successively insert the keys 10, 22, 31, 4, 15, 28, 17, 88, 59 into a hash table with 11 positions. Use the hash function* $h(k) = k \bmod 11$*, and resolve conflicts using open addressing with linear probing.*

(c) *Successively insert the keys 10, 22, 31, 4, 15, 29, 17, 88, 59 into a hash table with 11 positions. Resolve conflicts with cuckoo hashing, using* $h_1(k) = k \bmod 11$ *and* $h_2(k) = (k \bmod 13) \bmod 11$.

Show the individual steps.
Hint*: Pseudo-code for the hashing operations is available on the course website.*

Aufgabe im Anhang.